

This is my se lab 4.I want you to analyse the files, This is my github link(https://github.com/SETAPESU26/16_hangman).Help me solve the bugs in the game and also identify the bugs in the codes of every file in order to understand the working of the game and also help me fix it.

Yes. I checked both your **Lab 4 handout** and the actual `SETAPESU26/16_hangman` repository. The important thing is that this is **not a project where you should rewrite everything**—the README explicitly says to inspect the existing code, fix Tasks 1–4, test edge cases, and keep the project modular/in-memory.  [GitHub](#)

The Lab 4 handout also specifically requires you to reproduce the broken behaviour first, fix it using prompts, add the listed features, test before/after, and keep separate commits for tasks.

 PDF

 Lab_4_VibeCoding_Student_handou...

1. How your current game works

Your project has five important files:

File	Purpose	Current status
<code>main.py</code>	Starts the game	✅ Mostly correct
<code>game.py</code>	Main game logic	❌ Main bugs are here
<code>words.py</code>	Words, categories and hints	✅ Correct
<code>stats.py</code>	Session statistics	⚠️ Exists but isn't integrated
<code>requirements.txt</code>	Dependencies	✅ Correct

The repository README confirms this structure and describes the intended Tasks 1–4.  [GitHub](#)

2. BIGGEST BUG — repeated wrong guesses

This is the **Task 1 bug** you are supposed to reproduce.

Your current `guess()` function is:

</> Python

```
def guess(self, letter):
    if len(letter) != 1 or not letter.isalpha():
        return "Enter one letter."
    if letter in self.guessed:
        return "Already guessed."
    if letter in self.secret:
        self.guessed.add(letter)
        return "Correct."
    self.wrong.add(letter)
    self.lives -= 1
    return "Wrong."
```

The problem is here:

</> Python

```
if letter in self.guessed:
```

You check whether a letter was previously **correct**, but you don't check whether it was previously **wrong**.

So suppose the word is:

</> Plain text

```
python
```

You enter:

</> Plain text

```
z
```

You get:

</> Plain text

```
Wrong.
Lives: 5
```

Enter z again:

</> Plain text

Wrong.
Lives: 4

Again:

</> Plain text

Wrong.
Lives: 3

That's the bug.

The README specifically says a previously attempted wrong letter must not consume another life, and each distinct letter should affect the round exactly once. [GitHub](#)

Fix

Change:

</> Python

```
if letter in self.guessed:
```

to:

</> Python

```
if letter in self.guessed or letter in self.wrong:
```

So the corrected function becomes:

</> Python

```
def guess(self, letter):  
    if len(letter) != 1 or not letter.isalpha():  
        return "Enter one letter."  
  
    if letter in self.guessed or letter in self.wrong:  
        return "Already guessed."  
  
    if letter in self.secret:  
        self.guessed.add(letter)  
        return "Correct."  
  
    self.wrong.add(letter)
```

```
self.lives -= 1  
return "Wrong."
```

Why this works

Now there are three possibilities:

Invalid input

</> Plain text

abc

→ "Enter one letter."

No game state changes.

New correct letter

</> Plain text

p

→ added to guessed

No life lost.

New incorrect letter

</> Plain text

z

→ added to wrong

One life lost.

Previously used letter

Whether it was correct or wrong:

</> Plain text

p

p

or

</> Plain text

z
z

→ "Already guessed."

No state changes.

This directly satisfies Task 1.  GitHub

3. BUG — stats.py is never actually used

This is the second major issue.

Your stats.py contains:

</> Python

```
class SessionStats:
    def __init__(self):
        self.rounds = 0
        self.wins = 0
        self.best_streak = 0

    def record(self, won, streak):
        self.rounds += 1
        self.wins += int(won)
        self.best_streak = max(self.best_streak, streak)
```

The class itself is fine.

The problem is that **game.py** never imports or creates **SessionStats** .

So these values:

</> Python

```
rounds
wins
best_streak
```

never actually get updated.

The README explicitly requires integrating this supplied component so that rounds played, rounds won and best streak survive across multiple rounds.  [GitHub](#)

4. How the statistics should work

At the moment you have:

 Python

```
self.score = 0
self.streak = 0
```

in `HangmanGame` .

You should add:

 Python

```
from stats import SessionStats
```

and inside `__init__` :

 Python

```
self.stats = SessionStats()
```

So:

 Python

```
class HangmanGame:
    def __init__(self):
        self.score = 0
        self.streak = 0
        self.category = "technology"
        self.secret = ""
        self.guessed = set()
        self.wrong = set()
        self.lives = 6
        self.hint_used = False
        self.stats = SessionStats()
```

Now statistics belong to the **session**, not an individual round.

That's important.

5. Round state vs session state

This is one of the most important things to understand for your viva.

Round-specific state

These should reset when a new round begins:

</> Plain text

```
secret
guessed
wrong
lives
hint_used
```

Your `start_round()` already does this:

</> Python

```
def start_round(self):
    self.secret = random.choice(WORDS[self.category])
    self.guessed.clear()
    self.wrong.clear()
    self.lives = 6
    self.hint_used = False
```

That's mostly correct.

Session-specific state

These should NOT reset:

</> Plain text

```
score
streak
stats.rounds
stats.wins
stats.best_streak
```

Your current implementation keeps `score` and `streak`, which is good.

But `stats` isn't connected yet.

6. Another issue — difficulty doesn't exist

Task 3 says:

Add difficulty choices that alter the available lives and scoring.

Your current code always has:

```
</> Python
```

```
self.lives = 6
```

and:

```
</> Python
```

```
self.score += 5 + self.streak
```

So Easy, Medium and Hard don't exist.

The README requires difficulty to change round rules while not leaking state between rounds.



The README **doesn't specify exact Easy/Medium/Hard values**, so you shouldn't claim that particular values are mandated.

A reasonable implementation would be:

Difficulty	Lives	Base score
Easy	8	3
Medium	6	5
Hard	4	8

These are **proposed values**, not values specified by the repository.

7. Hint scoring problem

Current code:

</> Python

```
def use_hint(self):
    if self.hint_used:
        return None
    self.hint_used = True
    self.score = max(0, self.score - 1)
    return HINTS.get(self.secret, "No hint available.")
```

There are two things to understand.

Good

This prevents using the hint multiple times:

</> Python

```
if self.hint_used:
    return None
```

Good.

Problem

You're modifying the **overall session score** immediately:

</> Python

```
self.score = max(0, self.score - 1)
```

That makes scoring harder to reason about.

For example:

</> Plain text

Score = 20

New round

Use hint

Score = 19

Lose round

Score remains 19

A cleaner design is to make hint usage affect the **reward earned for that round**.

For example:

</> Python

```
round_score = base_score + streak_bonus

if self.hint_used:
    round_score -= 1

self.score += max(0, round_score)
```

This keeps the scoring logic tied to the completed round.

The README only says hint usage must affect scoring consistently, so this is a design choice rather than a hidden required value.  [GitHub](#)

8. Category handling

Your current category system is:

</> Python

```
print("\nCategories:", ", ".join(WORDS))
raw = input("Choose category or q: ").strip().lower()

if raw == "q":
    return

if raw not in WORDS:
    print("Unknown category.")
    continue

self.category = raw
```

This part is actually fairly good.

Your `words.py` contains:

</> Python

```
WORDS = {
    "technology": [...],
    "science": [...],
    "culture": [...]
}
```

So the category is validated against the actual dictionary.  [GitHub](#)

Minor improvement

You could make the interaction clearer:

</> Plain text

Available categories:

1. technology
2. science
3. culture

Choose category or q to quit:

But this isn't a major bug.

9. Invalid input

Your current validation:

</> Python

```
if len(letter) != 1 or not letter.isalpha():  
    return "Enter one letter."
```

is actually good.

For example:

</> Plain text

12

→ rejected

</> Plain text

ab

→ rejected

</> Plain text

@

→ rejected

</> Plain text

a

→ accepted

The important thing is that invalid input doesn't modify:

</> Plain text

```
lives
guessed
wrong
score
streak
```

Your current implementation satisfies that part.

The README specifically requires malformed input not to change game state.  [GitHub](#)

10. /hint and /quit

You already have command handling:

</> Python

```
if raw == "/quit":
    return False

if raw == "/hint":
    hint = self.use_hint()
    print(hint if hint else "Hint already used.")
    continue
```

This is structurally good.

However, because you use:

</> Python

```
raw = input(...).strip().lower()
```

these work:

</> Plain text

/HINT

/hint

/Hint

because everything is converted to lowercase.

Nice.

11. main.py

Your entire file is:

</> Python

```
from game import HangmanGame
```

```
if __name__ == "__main__":  
    HangmanGame().run()
```

There is no real bug here.

Its job is simply:

</> Plain text

main.py

↓

HangmanGame()

↓

run()

↓

game starts

So don't unnecessarily modify this file.

12. words.py

This file contains your game data:

</> Python

```
WORDS = {
    "technology": ["python", "compiler", "database", "algorithm", "network"],
    "science": ["gravity", "neuron", "galaxy", "enzyme", "volcano"],
    "culture": ["festival", "painting", "theatre", "heritage", "language"],
}
```

and hints:

</> Python

```
HINTS = {
    "python": "...",
    ...
}
```

This is functioning correctly.

One thing you should understand:

</> Python

```
self.secret = random.choice(WORDS[self.category])
```

means:

1. Get the selected category.
2. Get its list of words.
3. Randomly choose one.
4. Store it in `self.secret` .

For example:

</> Plain text

```
category = technology
    ↓
WORDS["technology"]
    ↓
["python", "compiler", ...]
    ↓
random.choice()
    ↓
"python"
```



```
self.secret
```

13. requirements.txt

This is also fine.

It says the project uses only the Python standard library and requires Python 3.9+.  [GitHub](#)

So **don't add unnecessary packages**.

This is also explicitly required by the README: no unnecessary external dependencies and no persistent storage.  [GitHub](#)

14. What I would change in game.py

Your current `game.py` is only 84 lines, so the best approach is **not** to completely replace the project.

I'd modify it around these four tasks:

Task 1

Fix repeated correct/wrong guesses.

Task 2

Integrate:

```
</> Python
```

```
SessionStats
```

Task 3

Add:

```
</> Plain text
```

```
Easy
```

```
Medium
```

```
Hard
```

with lives/scoring.

Task 4

Improve:

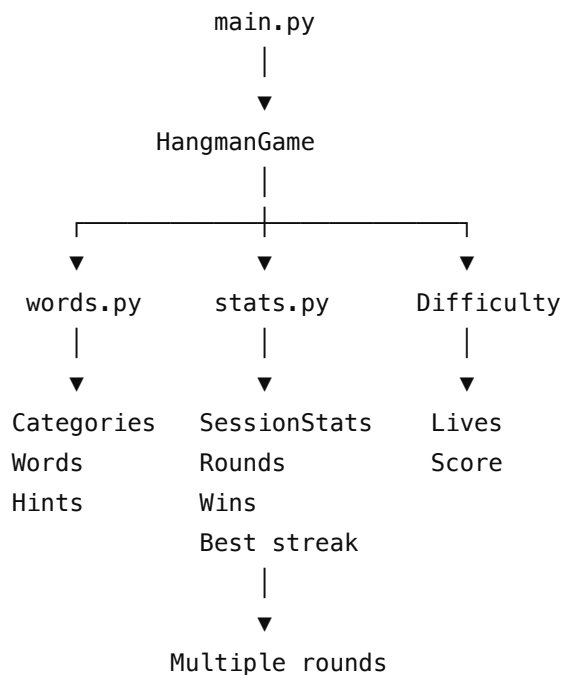
</> Plain text

```
invalid input
repeated commands
category selection
hint handling
quit
feedback
```

15. Final architecture after fixing

The game should look like this:

</> Plain text



And the state should be:

</> Plain text

```
SESSION
|
├─ score
├─ streak
├─ stats
│   └─ rounds
```

└─ wins

└─ best_streak

ROUND

|

└─ secret

└─ guessed

└─ wrong

└─ lives

└─ hint_used

└─ category

└─ difficulty

That separation is the key concept behind Task 2 and Task 3.

16. Bugs/features checklist for your Lab 4

You can use this as your implementation checklist:

Requirement	Current	Fix
Repeated correct guess	⚠️ Partially handled	Improve duplicate check
Repeated wrong guess	❌ Bug	Check wrong set
Invalid letter	✅	Keep
/hint	✅	Improve scoring
Repeated /hint	✅	Keep
/quit	✅	Keep
Category selection	✅	Minor feedback improvement
Session rounds	❌	Integrate SessionStats
Session wins	❌	Record completed rounds
Best streak	❌	Record streak
Difficulty	❌	Add
Difficulty lives	❌	Add

Requirement	Current	Fix
Difficulty scoring	✗	Add
Hint scoring	⚠	Make consistent
State reset	⚠	Separate round/session state
Feedback duplication	⚠	Clean command handling
Persistent storage	✅ None	Keep none
External dependencies	✅ None	Keep none

The repository itself lists these exact four task areas and the required testing categories.  [GitHub](#)

17. Most important: how you should do the assignment

Don't just paste a final solution and call it done.

Your Lab 4 specifically says to **use an LLM as a coding assistant while retaining responsibility for understanding and testing**, and to preserve the complete LLM chat history.  [GitHub](#)

So your workflow should be:

</> Plain text

```
1. Run original game
  ↓
2. Reproduce repeated-wrong-guess bug
  ↓
3. Ask LLM to identify/fix Task 1
  ↓
4. Test Task 1
  ↓
5. Commit Task 1
  ↓
6. Add SessionStats
  ↓
7. Test multiple rounds
  ↓
8. Commit Task 2
  ↓
9. Add difficulty/scoring
  ↓
10. Test all difficulties
```

The assignment also requires separate commits for each task.



Lab_4_VibeCoding_Student_handou...

One thing I would do next

First, don't modify anything yet. Run your original game and reproduce Task 1 exactly. For your 10-second **before** video, show something like:

</> Plain text

Choose category:
technology

Word: _ _ _ _ _
Lives: 6

Enter:
z

Wrong.
Lives: 5

Enter:
z

Wrong.
Lives: 4

That gives you clear evidence of the original defect.

Then we can fix **Task 1 first**, test it, and make your **first Git commit** before moving to Task 2. This matches the assignment's requirement to reproduce the defect before changing the code.



GitHub